

Unit:7

JAVA EXCEPTION HANDLING**Introduction:**

- An exception is an abnormal condition that arises in a code sequence at run time.
- In other words, an exception is a run-time error.
- In computer languages that do not support exception handling, errors must be checked and handled manually—typically through the use of error codes, and so on.

7.1 Handling Error and Exception, Catching Exception:**Error and Exception:**

S.N.	Error	Exception
1	An error is a problem that also arises during compile time of program which cannot be handled.	An exception is a problem that arises during the run time of a program which can be handled.
2	Recovering from Error is not possible.	Recovering is possible by either using try-catch block or throwing exceptions back to caller.
3	Errors are mostly caused by the environment in which program is running.	Program itself is responsible for causing exceptions.
4	All errors in java are unchecked type.	Exception are checked as well as unchecked type.
5	They are defined in java.lang.Error package.	They are defined in java.lang.Exception package
6	Examples: java.lang.StackOverflowError, java.lang.OutOfMemoryError	Examples: Checked Exceptions : SQLException, IOException UncheckedExceptions: NullPointerException,

❖ Handling of Exception in java:

- When in program the exception object is not caught properly the interpreter displays the exception message, to continue the program execution the exception should be catch which is thrown by exception object and display appropriate message for taking corrective action. This task is known as **Exception Handling**.
- The purpose of exception handling is to provide a means to detect and report an exceptional circumstance so that appropriate action can be taken.
- Java exception handling is managed by five keywords: **try, catch, throw, throws and finally**

The general syntax of exception handling block is:

```
try
{
    // block of code to monitor for errors
}
catch( Exception_type1 exob)
{
    //exception handler for exception type1
}
catch( Exception_type2 exob)
{
    //exception handler for exception type2
}
```

```
//.....
```

```
finally
```

```
{
```

```
    //block of code to be executed before try block ends
```

```
}
```

Here, exception type is the type of exception that has occurred.

1. **Try and catch:**

- The code that may generate exception is enclosed within try block.
- Immediately following try block, include a catch clause that specifies the exception type that you wish to catch.

Program which includes try and catch block that processes the Arithmetic Exception generated by the division-by-zero error.

```
class Exception
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        int a,b;
```

```
        try
```

```
        {
```

```
            b = 0;
```

```
            a = 12/b;
```

```
            System.out.println(" Inside try block");// this will not be printed
```

```
        }
```

```
        catch(ArithmeticException e)//catch division by zero error
```

```
        {
```

```
            System.out.println("Division by zero.");
```

```
        }
```

```
        System.out.println("Outside try and catch block");
```

```
    }
```

```
}
```

Output

Division by zero.

Outside try and catch block

1.1.**Multiple catch clauses:**

- ✓ In some cases, more than one exception could be raised by a single piece of code.
- ✓ To handle this type of situation, we can specify two or more **catch** clauses, each catching a different type of exception.
- ✓ When an exception is thrown, each catch statement is inspected in order, and the first one whose type matches that of the exception is executed.
- ✓ After one **catch** statement executes, the others are bypassed and execution continues after the **try/catch** block.

Syntax:

```
try
```

```
{
```

```
    Statements;
```

```
}
```

```
catch (exception_type1 e)
```

```
{  
    Statements;  
}  
catch (exception_type2 e)  
{  
    Statements;  
}  
catch (exception_typeN e)  
{  
    Statement;  
}
```

Write a program that demonstrate the concept of multiple catch blocks

class MultipleCatch

```
{  
    public static void main(String args[])  
    {  
        int a[] = {5,10};  
        int b = 5;  
        try  
        {  
            int x = a[2]/(b-a[1]);  
        }  
  
        catch(ArithmeticException e)  
        {  
            System.out.println("Division by zero");  
        }  
  
        catch(ArrayIndexOutOfBoundsException e)  
        {  
            System.out.println("Array index error");  
        }  
  
        catch(ArrayStoreException e)  
        {  
            System.out.println("Wrong data type");  
        }  
  
        int y = a[1]/a[0];  
        System.out.println("Y= " +y);  
    }  
}
```

Output

Array index error

Y=2

2. Throw and Throws:

No.	Throw	Throws
1)	Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
2)	Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
3)	Throw is followed by an instance.	Throws is followed by class.
4)	Throw is used within the method.	Throws is used with the method signature.
5)	Syntax: throw Instance //Example: throw new ArithmeticException (" / by zero");	return_type method_name() throws exception_class_name { //method code }
6)	You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method() throws IOException, SQLException.
7)	<u>Example of Throw:</u> <pre> public class GFG { public static void main(String[] args) { // Use of unchecked Exception try { // double x=3/0; throw new ArithmeticException(); } catch (ArithmeticException e) { e.printStackTrace(); } } } </pre> Output: java.lang.ArithmeticException: /by zero at UseOfThrow.main(UseOfThrow.java:8)	<u>Example of Throws:</u> <pre> { //some statements } throws: import java.io.IOException; public class UseOfThrowAndThrows { public static void main(String[] args) throws IOException { } } </pre> Output: Exception in thread "main" java.io.IOException at UseOfThrowAndThrows.main(UseOfThrow.java:7)

3. Finally:

- Java supports another statement known as **finally** statement that can be used to handle an exception that is not caught by any of the previous **catch** statements.
- **finally** block can be used to handle any exception generated within a **try** block.
- It may be added immediately after the **try** block or after the last **catch** block.

Syntax 1

```
try
{
    .....
}
finally
{
    .....
}
```

Example:

```
class MultipleCatch
{
    public static void main(String args[])
    {
        int a[] = {5,10};
        int b = 5;
        try
        {
            int x = a[2]/(b-a[1]);
        }
        catch(ArithmeticException e)
        {
            System.out.println("Division by zero");
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println("Array index error");
        }
        catch(ArrayStoreException e)
        {
            System.out.println("Wrong data type");
        }
        finally
        {
            int y = a[1]/a[0];

            System.out.println("Y= " +y);
        }
    }
}
```

Syntax 2

```
try
{
    .....
}
catch(.....)
{
    .....
}
catch(.....)
{
    .....
}
finally
{
    .....
}
```

7.2. Types of Exception:

There are three types of Exceptions. They are described below:

1. Checked Exception:

- They are the exceptions that are checked at compile time.
- If some code within a method throws a checked exception, then the method must either handle the exception or it must specify the exception using **throws** keyword.

Example:

```
import java.io.*;
class Example {
    public static void main(String args[])
    {
        FileInputStream fis = null;
        fis = new FileInputStream("B:/myfile.txt");
        int k;
        while(( k = fis.read() ) != -1)
        {
            System.out.print((char)k);
        }
        fis.close();
    }
}
```

Output:

Exception in thread "main" java.lang.Error: Unresolved compilation problems:
Unhandled exception type FileNotFoundException
Unhandled exception type IOException
Unhandled exception type IOException

2. Unchecked Exception:

- They are the exceptions that are not checked at compiled time.
- In java, all exceptions are unchecked, so it is not forced by the compiler to either handle or specify the exception. These are also called as Runtime Exceptions
- It is up to the programmers to be civilized, and specify or catch the exceptions.
- In Java exceptions under Error and Runtime Exception classes are unchecked exceptions, everything else under throwable is checked.

Example:

```
public class Unchecked_Demo {
    public static void main(String args[]) {
        int num[] = {1, 2, 3, 4};
        System.out.println(num[5]);
    }
}
```

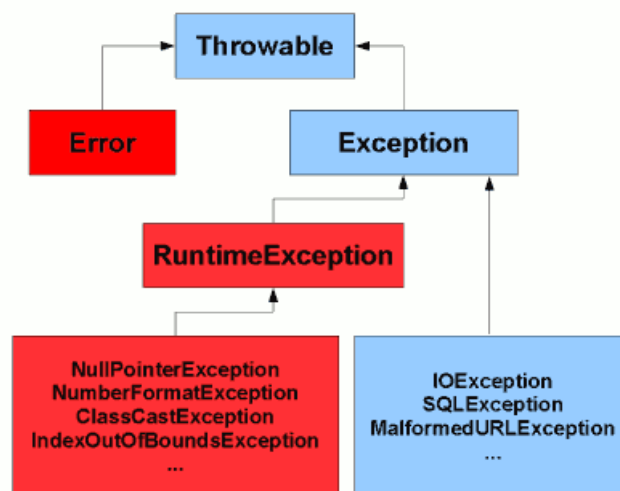
Output:

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 5
at Exceptions.Unchecked_Demo.main(Unchecked_Demo.java:8)

3. Error:

- Errors are also unchecked exception & the programmer is not required to do anything with these.
- These are not exceptions at all, but problems that arise beyond the control of the user or the programmer.
- Most often, recovery from an Error is not possible & the program should be allowed to terminate.
- They are also ignored at the time of compilation.
- Examples include OutOfMemoryError, StackOverflowError, etc.

❖ Throwable:



- All exception types are subclasses of the built-in class **Throwable**.
- Thus, Throwable is at the top of the exception class hierarchy.
- Immediately below Throwable are two subclasses that partition exceptions into two distinct branches.
- One branch is headed by *Exception* and another branch is headed to *error*.

Class declaration:

Following is the declaration for java.lang. Throwable class:

```
public class Throwable
    extends Object
    implements Serializable
```

Example:

```
{
//Customized Exception class
}
class ThrowAndThrowsExample
{
void method() throws MyException
{
MyException e = new MyException();
throw e;
}
}
```

Example of user defined Exception (imp for exam):

/* This is my Exception class, I have named it MyException

```
class MyException extends Exception{
    String str1;
    MyException(String str2) {
        str1=str2;
    }
    public String toString()
    {
        return ("MyException Occurred: "+str1) ;
    }
}

class Example1{
    public static void main(String args[]){
        try{
            System.out.println("Starting of try block");
            // I'm throwing the custom exception using throw
            throw new MyException("This is My error Message");
        }
        catch(MyException exp){
            System.out.println("Catch Block") ;
            System.out.println(exp) ;
        }
    }
}
```

Output:

Starting of try block

Catch Block

MyException Occurred: This is My error Message

Example of Try catch and finally:[Imp for Exam]

```
public class TestFinallyBlock2{
    public static void main(String args[]){
        try{
            int data=25/0;
            System.out.println(data);
        }
        catch(ArithmeticException e){System.out.println(e);}
        finally{System.out.println("finally block is always executed");}
        System.out.println("rest of the code...");
    }
}
```

Output:

Exception in thread main java.lang.ArithmeticException:/ by zero
finally block is always executed
rest of the code...

7.2 Debugging Technique**7.2.1. Loop Bug**

- The two most common kinds of loop errors are unintended *infinite loops* and *off-by-one errors*.

Infinite Loop: A loop that runs forever is called an infinite loop.

- A while loop, do-while loop, or for loop does not terminate as long as the controlling Boolean expression evaluates to true.
- This Boolean expression normally contains a variable that will be changed by the loop body, and usually the value of this variable eventually is changed in a way that makes the Boolean expression false and therefore terminates the loop.
- However, if we make a mistake and write our program so that the Boolean expression is always true, then the loop will run forever.

Off-by-one error: The loop repeats the loop body *one too many or one too few times*.

- These sorts of errors can result from carelessness in designing a controlling Boolean expression.
- For example, if we use less-than when we should use less-than-or-equal, this can easily make our loop iterate the body the wrong number of times.
- Use of == to test for equality in the controlling Boolean expression of a loop can often lead to an off-by-one error or an infinite loop.

7.2.2. Tracing Variables:

- One good way to discover errors in a loop or any kind of code is to trace some key variables.
- Tracing variables means watching the variables change value while the program is running.
- Most programs do not output each variable's value every time the variable changes, but being able to see all of these variable changes can help us to debug your program.

7.3. General Debugging Techniques:

- Tracing errors can sometimes be a difficult and time-consuming task.
- If we are having difficulties finding the source of our errors, then there are some general debugging techniques to consider.
- If the program is giving incorrect output values, then we should examine the source code, different test cases using a range of input and output values, and the logic behind the algorithm itself.
- Determining the precise cause and location of a bug is one of the first steps in fixing the error.
- Examine the input and output behaviour for different test cases to try to localize the error.
- A related technique is to trace variables to show what code the program is executing and what values are contained in key variables.
- We might also focus on code that has recently changed or code that has had errors before. Finally, we can also try removing code.
- The process can be repeated until the location of the error can be pinpointed.
- The first mistakes we should look for are common errors that are easy to make.
- Examples of common errors include off-by-one errors, comparing floating-point types with ==, adding extra semicolons that terminate a loop early, or using == to compare strings for equality.

7.4. Stream, Zip file stream, object Stream and Handling Files:

- A stream is linked to a physical device by the Java I/O system.
- All the streams behave in the same manner, even if the actual physical devices to which they are linked differ.
- Thus, the same I/O classes and the methods can be applied to the different types of devices

Types of Stream

Java defines the following two types of streams:

Byte streams

Character streams

❖ Java Byte Stream:

- The Byte streams provide a convenient means for handling input and output of bytes. Byte streams are used, for example, when reading or writing binary data.
- Byte streams are defined by using two hierarchies.
- At the top are the following two abstract classes:

✓ InputStream:

Example : BufferedInputStream, ByteArrayInputStream, FileInputStream etc.

✓ OutputStream:

Example: BufferedOutputStream, ByteArrayOutputStream, FileOutputStream etc.

❖ Java Character Stream:

- The Character stream provide a convenient means for handling input and output of characters.
- They are Unicode and, therefore, can be internationalized. Also, in some cases, character streams are more efficient than the byte streams.
- Character streams are defined by using the two class hierarchies. At the top are two abstract classes:

✓ **Reader:**

Example: BufferedReader, CharArrayReader, FileReader, InputStreamReader etc.

✓ **Writer:**

Example: BufferedWriter, CharArrayWriter, FileWriter, OutputStreamWriter etc.

Important Questions from this unit for exams:

Q.1) what is difference between error and exception? Write a program to demonstrate user defined exception/custom exception/owner class exception.

Q.2) differentiate between throw, throws and throwable with examples?

Q.3) Define exception. How it can be handled with example.